

From Similarity Ranking to Definitive Verdict: LLM-Enhanced Source-to-Binary Function Localization

ANONYMOUS AUTHOR(S)

Locating a known source function in a stripped binary is a prerequisite for many security and software engineering tasks, including Software Composition Analysis (SCA) false-positive elimination, patch presence verification, malware analysis, code plagiarism detection, and license compliance auditing. We formalize this need as *source-to-binary function localization*: given the source code of a target function and its encompassing source package, determine whether the function is present in a stripped binary and, if so, report its address. Two fundamental challenges arise: *cross-modal alignment*, as source code and stripped binary reside in vastly different representation spaces; and *similar function disambiguation*, as compilation erases the symbolic features that distinguish functionally similar functions.

We present XLoc, a recall-then-verify framework built on two insights. First, cross-modal alignment does not require costly and error-prone compilation; it only demands token-level alignment, a process that can be reliably approximated. Second, the information needed to disambiguate similar functions is already available on the source side and can be extracted ahead of time to guide verification. Building on these insights, XLoc implements a multi-stage recall module in which an LLM transforms source code into pseudo-decompiled representations aligned with binary decompilation output, bridging the cross-modal gap. For verification, XLoc identifies potentially confusing similar functions, extracts differential summaries, and uses them to guide the verification process toward the specific distinguishing evidence for each candidate, producing definitive accept/reject verdicts rather than similarity rankings.

We evaluate XLoc on two complementary datasets spanning 196 CVEs, 480 vulnerable functions, and 756 binaries. XLoc achieves up to 84.4% localization accuracy (4.2× over the best baseline) and HM = 87.1% for positive/negative discrimination (vs. 35.1% for the best baseline). These results demonstrate that XLoc can locate target functions with high accuracy, reliably discriminate between positive and negative cases, and produce definitive verdicts.

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Software and its engineering** → **Software libraries and repositories**.

Additional Key Words and Phrases: Static Binary Analysis, Function Localization

ACM Reference Format:

Anonymous Author(s). 2018. From Similarity Ranking to Definitive Verdict: LLM-Enhanced Source-to-Binary Function Localization. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 26 pages. <https://doi.org/XXXXXXX.XXXXXXX>.

1 Introduction

As software supply chains grow increasingly complex, known vulnerabilities in third-party open-source components propagate along dependency chains to vast numbers of downstream products, making vulnerability detection in deployed binaries a critical security need [13, 43]. Binary Software Composition Analysis (SCA) tools address this by identifying which components and versions a

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

binary incorporates, flagging known vulnerabilities such as “this binary includes OpenSSL 1.0.1, which contains CVE-2014-0160” [25, 27, 42, 50]. However, a component-level finding does not imply that the specific vulnerable function is actually present in the binary: partial reuse, conditional compilation, module-level linking decisions, and version-specific code changes can all cause the vulnerable function to be absent [11, 27, 54], leading to false positives. More broadly, confirming whether a specific function exists in a stripped binary and pinpointing its address is a foundational capability that extends well beyond SCA false-positive elimination: it equally underpins security and software engineering tasks such as patch presence verification [10, 28, 57], function-level reachability and exploitability assessment [8, 51], malware analysis [24, 34], code plagiarism detection [58], and license compliance auditing [14]. All of these tasks require reliably locating a target source function in a stripped binary.

We formalize this need as *source-to-binary function localization*: given the source code of a target function together with its encompassing source package, and a stripped target binary, determine whether the function exists in the binary and, if so, report its address.

Existing methods applicable to this task follow two strategies. *Source-to-binary* methods directly compare source and binary representations: BinPro [35] and B2SFinder [56] rely on manually crafted features such as string constants and numeric literals; CodeCMR [55] trains cross-modal encoders to project source and binary into a shared embedding space. *Binary-to-binary* methods [19, 33, 41, 44, 46] avoid the cross-modal difficulty by first compiling the source into a reference binary and then comparing within the binary modality. Both strategies produce similarity-ranked lists rather than definitive verdicts, and face two fundamental challenges:

Challenge 1: Cross-Modal Alignment. Source code and binary are fundamentally different modalities: compilation is a *many-to-many* mapping where a single source function yields vastly different binaries depending on compiler, optimization level, and target architecture, while syntactically different source functions with equivalent semantics can produce similar or identical binaries after compilation. The source-to-binary methods above struggle with this gap: manually crafted features are absent in many functions, and learned encoders cannot reliably align unstable binary-side representations. Binary-to-binary methods face the same compilation barrier and additionally demand a complete dependency chain, incurring high compilation overhead and frequent build failures [21].

Challenge 2: Similar Function Disambiguation. Real-world codebases contain abundant similar functions that share nearly identical structure yet differ in subtle ways, such as API families, protocol variants, and wrapper functions that dispatch to different types or callees. Producing a definitive match verdict requires disambiguating these similar functions, yet this is inherently difficult because compilation and stripping blur the very symbolic features (function names, type names) that distinguish them. And this loss of discriminative information leads to the failure of existing approaches.

Both challenges stem from the same root cause: compilation systematically transforms source code, erasing the features needed for cross-modal alignment and for distinguishing similar functions. We observe that these lost features can be recovered through source-side analysis, since the source retains the information that compilation removes. This leads to two key insights.

Insight 1: Token-level simulation of compilation suffices for cross-modal alignment. Since similarity matching does not require source-side semantic equivalence but only token-level proximity, we do not need to actually compile source code to align it with binary. Instead, it suffices to transform the tokens and token sequences in the source into a form that resembles real decompiled code, which can then be directly aligned with the decompiled code produced from the target binary side by decompilers. Moreover, the transformations from source to decompiled code

(symbol erasure, type degradation, stylistic changes) are systematic and predictable, making such simulation feasible.

Insight 2: The knowledge needed to distinguish similar functions is preserved at the source level and can guide binary-side discrimination. Each pair of similar functions may diverge in its own way: some are distinguishable by their callees, others by the struct types they access, and so on. These function-specific distinguishing features are clearly visible at the source level, where symbols remain intact. Compilation obscures them in the binary, burying them among other noise, yet they are not destroyed. As long as binary-side discrimination focuses on these key distinguishing points, similar functions can still be told apart. Since such distinguishing points can be learned from source code, source-level analysis can guide binary-side discrimination.

Based on these insights, we propose XLoc, a *recall-then-verify* funnel architecture that addresses compilation-induced information loss at successive levels of granularity. The recall stage achieves high coverage across diverse compilation configurations, while the verify stage precisely disambiguates similar candidates, ultimately producing a definitive verdict: a confirmed address or a justified conclusion of absence. In the recall stage, two lightweight methods first exploit features that survive compilation (preserved symbols, unique strings, call-graph edges, file proximity) for fast matching or candidate narrowing. When these features are insufficient, an LLM-based strategy simulates compilation effects on the source to produce a pseudo-decompiled representation while a decompiler lifts binary functions (**Insight 1**), converging both sides into the same feature space for effective same-modal retrieval. In the verification stage, an LLM acts as a reasoner: it first narrows down the recalled candidates through coarse screening, then applies tournament-style pairwise comparison, producing a definitive verdict rather than a similarity ranking. When similar candidates prove difficult to distinguish, pre-constructed diff summaries direct the LLM’s attention to the specific discriminative evidence (callees, struct usages, etc.) for each pair (**Insight 2**).

While XLoc’s technique is general-purpose, we evaluate it in the vulnerability localization setting, which represents the most pressing practical demand. We conduct experiments on two complementary datasets covering 196 CVEs, 480 vulnerable functions, and 756 binaries. *Lares* [28] (5,448 positive cases) contains binaries compiled under diverse architectures, optimization levels, and compilers; *Debian* (513 positive + 2,010 negative cases) additionally includes binaries from the same project that do not contain the target function, testing the ability to produce absence verdicts. Compared with four baselines spanning source-to-binary matching (CodeCMR [55]), binary similarity (DeJina [41]), and general-purpose code embeddings (UniXCoder [18], Jina-0.5B [26]), XLoc achieves the highest localization accuracy on *Lares* (84.4%, 4.2× over the best baseline) with stable performance across all configurations, and the strongest positive/negative discrimination on *Debian* (HM = 87.1% vs. 35.1% for the best baseline). Ablation studies further confirm the effectiveness of each proposed component.

Our main contributions are as follows:

- **Token-level pseudo-decompilation for same-space recall.** We propose a recall strategy that uses an LLM to simulate compilation effects on source code at the token level, producing a pseudo-decompiled representation that aligns with decompiler output from the binary side. This transforms the cross-modal source-to-binary retrieval problem into a same-modal comparison without requiring actual compilation or build infrastructure, substantially boosting existing embedding models (e.g., R@1: 19.3% → 55.1%).
- **Source-guided tournament verification for similar function disambiguation.** We propose a verification framework that combines tournament-style LLM reasoning with source-level differential knowledge to disambiguate similar functions. An offline diff summary mechanism pre-extracts the discriminative features between similar functions from the source and directs

the LLM’s attention to the corresponding evidence in the binary, producing a confirmed address or an absence verdict (false positive rate: 2.8%).

- **Framework.** We integrate the above components into XLoc, an end-to-end source-to-binary function localization system. The online pipeline runs on a single 40 GB GPU using open-source models/LLMs, enabling private deployment.
- **Evaluation.** We evaluate XLoc on two datasets (196 CVEs, 480 vulnerable functions; 5,448 positive and 2,010 negative cases) under diverse compilation configurations. XLoc achieves 84.4% localization accuracy (4.2× over the best baseline) and HM = 87.1% for positive/negative discrimination, outperforming the best baseline by 52.0 pp.

2 Background and Motivation

2.1 Problem Formulation

As outlined in § 1, the task of source-to-binary vulnerable function localization takes three inputs: the source code of a vulnerable function f_{src} , the source code repository $\mathcal{R}$ from which f_{src} originates, and a target binary B containing functions $F_B = \{f_1, f_2, \dots, f_n\}$. The goal is to compute:

$$Localize(f_{src}, \mathcal{R}, B) \rightarrow (found, addr) \quad (1)$$

where $found \in \{\text{true}, \text{false}\}$ indicates whether f_{src} is present in B , and $addr \in Addr(B) \cup \{\perp\}$ gives the matching function’s address when $found = \text{true}$, or $\perp$ otherwise.

2.2 Motivation

We illustrate the challenges of this problem through a concrete vulnerability from OpenSSL. CVE-2014-8275 is a certificate fingerprint validation bypass that affects multiple signature verification routines in the library. Figure 1 shows two functions impacted by this CVE: DSA_verify and ECDSA_verify. In practice, if a vulnerability scanner fails to locate DSA_verify in a target binary, the affected binary goes unreported (false negative); conversely, if the scanner confuses the two functions and reports ECDSA_verify as DSA_verify, the resulting mismatch can mislead triage and patching efforts (wrong positive). These failure modes motivate a localization approach that is both *recall-capable* and *verification-precise*.

Cross-Modal Alignment. Compilation erases symbolic information (function names, variable names, type annotations) and restructures control flow through optimization passes, producing a binary representation that bears little resemblance to the original source. As shown in Figures 1 and 2, DSA_verify’s decompiled output retains none of the original identifiers, with all names replaced by addresses and types degraded to generic `__int64`. Existing feature-based methods [35, 56] fail when no distinctive strings or constants survive; cross-modal embedding methods [55] struggle because compilation is a many-to-many mapping that prevents reliable source-to-binary alignment. In our evaluation, neither approach recalls DSA_verify among its top candidates.

Similar Function Disambiguation. Even when the target function is successfully recalled, producing a definitive verdict requires disambiguating it from similar functions. As shown in Figures 1 and 2, DSA_verify and ECDSA_verify share identical control flow and differ only in type names and callee prefixes, all of which are erased during compilation and stripping, leaving nearly identical decompiled code with all distinguishing names replaced by opaque addresses. Existing methods apply a uniform feature extraction scheme to all functions, inherently unable to adapt to the specific differences that matter in each case; in our experiments, embedding-based methods rank DSA_verify above ECDSA_verify in only ~50% of queries, equivalent to random guessing. This inability to disambiguate them also undermines absence determination: when a binary contains ECDSA_verify but not DSA_verify, the near-identical structure causes the former

```

1 int DSA_verify(int type,
2   const unsigned char *dgst, int dgst_len,
3   const unsigned char *sigbuf,
4   int siglen, DSA *dsa) {
5   DSA_SIG *s; int ret = -1;
6   s = DSA_SIG_new();
7   if (s == NULL) return(ret);
8   if (d2i_DSA_SIG(&s, &sigbuf,
9     siglen) == NULL) goto err;
10  ret = DSA_do_verify(dgst,
11    dgst_len, s, dsa);
12 err: DSA_SIG_free(s);
13  return(ret);
14 }

```

(a) DSA_verify (target)

```

1 int ECDSA_verify(int type,
2   const unsigned char *dgst, int dgst_len,
3   const unsigned char *sigbuf,
4   int siglen, EC_KEY *eckey) {
5   ECDSA_SIG *s; int ret = -1;
6   s = ECDSA_SIG_new();
7   if (s == NULL) return(ret);
8   if (d2i_ECDSA_SIG(&s, &sigbuf,
9     siglen) == NULL) goto err;
10  ret = ECDSA_do_verify(dgst,
11    dgst_len, s, eckey);
12 err: ECDSA_SIG_free(s);
13  return(ret);
14 }

```

(b) ECDSA_verify (similar)

Fig. 1. Two OpenSSL functions affected by CVE-2014-8275. They share identical control flow and differ only in the **highlighted** type names and callee prefixes, all of which are removed during compilation and stripping.

```

1 __int64 sub_FD377(
2   __int64 a1, __int64 a2,
3   unsigned int a3, __int64 a4,
4   int a5, __int64 a6) {
5   v12 = a4;
6   ptr[1] = __readfsqword(0x28u);
7   ptr[0] = sub_FD445();
8   if (ptr[0]) {
9     v9 = sub_FD246(ptr, &v12, a5);
10    v10 = -1;
11    if (v9)
12      v10 = sub_FD415(a2, a3,
13        ptr[0], a6);
14    sub_FD478(ptr[0]);
15  } else return -1;
16  return v10;
17 }

```

(a) sub_FD377 (DSA_verify)

```

1 __int64 sub_FE218(
2   __int64 a1, __int64 a2,
3   unsigned int a3, __int64 a4,
4   int a5, __int64 a6) {
5   v12 = a4;
6   v13[1] = __readfsqword(0x28u);
7   v13[0] = sub_190B92();
8   if (v13[0]) {
9     v9 = sub_190B68(v13, &v12, a5);
10    v10 = -1;
11    if (v9)
12      v10 = sub_FE1D0(a2, a3,
13        v13[0], a6);
14    sub_190BA7(v13[0]);
15  } else return -1;
16  return v10;
17 }

```

(b) sub_FE218 (ECDSA_verify)

Fig. 2. Decompiled output of the two functions from Figure 1. After compilation and stripping, the two become nearly identical; the only differences are the raw callee addresses (**highlighted**).

to score high against the latter’s query, and no reliable threshold can separate true matches from such false positives (oracle-optimal FPR: 33.4%).

The two challenges above expose fundamental limitations of existing similarity-based approaches: cross-modal alignment prevents reliable recall, and uniform feature extraction cannot disambiguate similar functions, making definitive verdicts unattainable. These challenges motivate the design of XLoc (§ 3).

3 Approach

To address the challenges, we propose XLoc (Figure 3). To address *cross-modal alignment*, XLoc uses an LLM to simulate compilation effects on the query source function at the token level, producing a pseudo-decompiled representation, while lifting binary functions via decompilation, converging both sides into the same representation space for retrieval. To address *similar function disambiguation*, XLoc introduces LLM-based reasoning for verification, and designs an offline source-level analysis that pre-extracts how the target function differs from its similar counterparts; at query time, this differential knowledge guides the LLM to identify specific discriminative evidence in the binary.

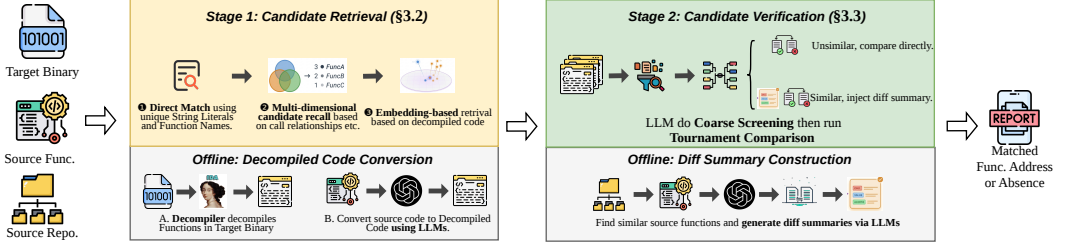


Fig. 3. Overall architecture of XLoc.

Concretely, XLoc takes three inputs: the target function source code f_{src} , the source repository $\mathcal{R}$ from which it originates, and a target binary B , and operates in two stages. **The recall stage (§ 3.2)** narrows the search space from tens of thousands of binary functions to tens through a hierarchical strategy: it first attempts exact match via preserved symbols and unique strings, then scores candidates by compilation-surviving features (shared strings, call-graph edges, file proximity), and finally falls back to LLM-bridged embedding retrieval in the unified decompiled-code representation for functions lacking distinctive features. **The verification stage (§ 3.3)** employs the LLM as a reasoner: it performs coarse screening followed by tournament-style pairwise comparison, with source-level diff summaries injected on demand to disambiguate similar functions, ultimately producing a definitive verdict: a confirmed address if a candidate survives, or an absence conclusion if all are eliminated.

3.1 Preprocessing

Before recall and verification can begin, XLoc extracts structured information from both the source repository and the target binary.

Source-Side Preprocessing. Given the source repository $\mathcal{R}$, XLoc performs a one-time analysis that is independent of any specific target binary. For each function in $\mathcal{R}$, it extracts the function body, the set of string literals, and the caller/callee relationships from the call graph. It also records each function’s source file path and the number of functions per file, which are used later for file-proximity-based candidate selection (§ 3.2.2). In addition, XLoc precomputes *diff summaries* that capture source-level differences between similar functions, for use during verification; the construction process is detailed in § 3.3.3.

Binary-Side Preprocessing. For each target binary B , XLoc decompiles all functions to obtain pseudocode representations, which serve as input to both the embedding-based recall (§ 3.2.3) and candidate verification (§ 3.3). It also extracts string literals from data sections and builds a cross-reference index mapping each string to the functions that reference it, recovers the binary-level call graph, collects preserved symbols from import/export tables, and records the address-ordered function list. These artifacts provide the feature dimensions used by rule-based recall (§ 3.2.2).

3.2 Hierarchical Candidate Recall

XLoc adopts a hierarchical recall strategy that progressively increases effort. *Exact match* first attempts to resolve the query immediately via preserved symbols or unique string literals. When no definitive match is found, *multi-dimensional candidate selection* (§ 3.2.2) scores binary functions by compilation-surviving features (shared strings, call-graph edges, file proximity) to narrow the search space without model inference. For functions lacking such distinctive features, *LLM-bridged embedding recall* (§ 3.2.3) transforms the source function into a binary-like representation, reducing cross-modal retrieval to same-space embedding search.

3.2.1 Exact Match. XLoc first attempts to identify the target by exploiting distinctive features of the source function that survive compilation intact: (a) *symbol match*: even in stripped binaries, exported function names are retained in import/export tables; if the function name of f_{src} appears in B 's preserved symbol table, it is matched directly; or (b) *unique string match*: a string literal that occurs exclusively in f_{src} within $\mathcal{R}$ is referenced by exactly one function in B . If either approach succeeds, XLoc reports the matched function directly and terminates.

3.2.2 Multi-dimensional Candidate Selection.

When exact match does not resolve f_{src} , XLoc selects candidates by systematically exploiting features that survive compilation. We focus on three categories of compilation-surviving features: string literals (preserved in data sections), call relationships (call instructions survive even when names are stripped), and file-level spatial locality (compilers typically emit functions in source-file order, so same-file functions occupy contiguous addresses [25]). We exclude numeric constants (used by previous works [55, 56]), as they are unreliable across compilation: they can be confused with memory-access offsets from struct lowering, and optimizations may alter their values (e.g., $a > 16 \rightarrow a >= 15$).

To leverage call relationships and spatial locality, XLoc first locates *anchor points* in B using the same symbol and unique-string matching techniques from the exact-match step. We distinguish two types of anchors: (i) *call-graph anchors*: callees or callers of f_{src} in the source-level call graph that can be located in B (e.g., exported functions whose names are preserved in import/export tables); and (ii) *same-file anchors*: functions residing in the same source file as f_{src} that can be similarly located, or binary functions that reference a string containing the source file path of f_{src} (e.g., the C standard requires that `assert()` embeds `__FILE__` into the binary [1], and many project-specific logging macros follow the same convention).

Based on these anchors, XLoc selects candidates along two dimensions. (1) *Shared call-graph neighbors*: any binary function that calls a callee anchor of f_{src} , or is called by a caller anchor of f_{src} . (2) *File proximity*: given a same-file anchor g , XLoc defines a proximity neighborhood:

$$N(g, x) = \{ f_i \in F_B \mid |idx(f_i) - idx(g)| \leq x - 1 \} \quad (2)$$

where $idx(f)$ is f 's index in the address-sorted function list and x is the number of functions in f_{src} 's source file. If multiple anchors are available, the candidate set is narrowed by intersection: $C_{prox} = \bigcap_{g \in G_{anchor}} N(g, x)$. Additionally, XLoc exploits *shared string literals* as an anchor-independent signal: string constants are preserved verbatim in data sections, so any binary function referencing the same strings as f_{src} is a candidate regardless of whether anchors are available. Each candidate f_i is scored by the total number of feature hits across all three dimensions:

$$score(f_i) = \underbrace{|S(f_{src}) \cap S(f_i)|}_{\text{shared strings}} + \underbrace{|CG_A(f_{src}) \cap CG_A(f_i)|}_{\text{call-graph neighbors}} + \underbrace{\mathbb{1}[f_i \in C_{prox}]}_{\text{file proximity}} \quad (3)$$

where $S(f)$ denotes the set of string literals in f , $CG_A(f)$ denotes f 's anchored call-graph neighbors (callees and callers), and $\mathbb{1}[\cdot]$ is the indicator function (1 if true, 0 otherwise). Intuitively, a candidate supported by more feature dimensions is more likely to be the target. The top-ranked candidates form C_{rule} for subsequent verification.

3.2.3 LLM-Bridged Embedding recall. For functions that lack distinctive local features (unique strings, anchored callees, or file-proximity signals), the preceding exact match and candidate selection steps are ineffective. To handle these cases, XLoc uses an LLM to transform the source function into a pseudo-decompiled representation while lifting binary functions via decompilation, converging both sides into the same decompiled-code space; an embedding model then performs same-space retrieval to select candidates.

```

1 int DSA_verify(int type,
2   const unsigned char *dgst, int dgst_len,
3   const unsigned char *sigbuf,
344   int siglen, DSA *dsa) {
345   DSA_SIG *s; int ret = -1;
346   s = DSA_SIG_new();
347   if (s == NULL) return(ret);
348   if (d2i_DSA_SIG(&s, &sigbuf,
349     siglen) == NULL) goto err;
350   ret = DSA_do_verify(dgst,
351     dgst_len, s, dsa);
352   err: DSA_SIG_free(s);
353   return(ret);
354 }

```

(a) Source code

```

1 __int64 __fastcall sub_401000(int a1,
2   __int64 a2, int a3,
3   __int64 a4, int a5, __int64 a6) {
4   v6 = DSA_SIG_new();
5   result = 0xFFFFFFFFLL;
6   if ( v6 ) {
7     v7 = d2i_DSA_SIG(&v6, &a4, a5);
8     if ( v7 )
9       result = (int)DSA_do_verify(
10         a2, a3, v6, a6);
11     DSA_SIG_free(v6);
12   }
13   return result;
14 }

```

(b) Pseudo-decompiled (LLM-generated)

Fig. 4. Source code of DSA_verify and its LLM-generated pseudo-decompiled representation.

Step 1: LLM-Based Representation Conversion. Converting source code to decompiled form normally requires compilation and decompilation, but compilation demands a complete build environment and dependency chain, with reported success rates as low as 10%–71% [21]. Our key observation is that downstream embedding retrieval operates at the token level: it suffices for the transformed representation to be structurally and lexically similar to real decompiler output, without semantic equivalence. This relaxation makes it feasible to use an LLM to approximate the combined effect of compilation and decompilation directly at the source level.

We analyze the appearance divergence between source code and decompiled output and address it with four transformation steps: (1) *compile-time code expansion* handles preprocessor divergence (macros, conditional compilation); (2) *compilation optimization* handles optimization divergence (constant folding, dead code elimination, control-flow restructuring); (3) *symbol stripping* handles information-loss divergence (name erasure, type degradation); (4) *stylization* handles decompiler-style divergence (naming conventions, formatting). Steps (1)–(2) approximate the structural changes of compilation, while (3)–(4) align token-level appearance with decompiled output.

(1) *Compile-time code expansion.* Macros, conditional directives, and compile-time expressions (e.g., `sizeof`) are expanded during compilation, causing structural divergence. We extract the relevant macro definitions, build configurations, and type definitions from $\mathcal{R}$ and supply them as context, enabling the LLM to perform the same expansions at the source level.

(2) *Compilation optimization.* Since our goal is surface-level similarity rather than semantic equivalence, we focus on lightweight optimizations that most commonly appear in decompiled code: constant propagation and folding, dead code elimination, and control-flow restructuring (e.g., collapsing `goto`-based error handling into nested conditionals).

(3) *Symbol stripping.* Variable and parameter names are unconditionally stripped. Function names require finer treatment: for each callee c of f_{src} , we classify it as *imported* (no definition in $\mathcal{R}$), *exported* (defined, declared in a header, non-static), or *internal*; only imported and exported names are retained. For types, we supply struct definitions from $\mathcal{R}$ so the LLM can convert field accesses into offset-style operations (e.g., `obj->field` $\rightarrow$ `*(a1 + 0x18)`) and degrade type annotations to primitives; exact byte-level offsets are not required, as they depend on compiler-specific layout decisions (alignment, padding), and our goal is structural resemblance rather than precise values.

(4) *Stylization.* We provide few-shot examples of decompiler output to guide the LLM in mimicking naming conventions (e.g., `v1`, `a1`), type representations, and formatting. In practice, all four steps are structured as a chain-of-thought prompt (Figure 5), with preprocessed context (macro definitions, struct layouts, callee classifications, and decompiler examples) injected as input.

Prompt Template: Source-to-Pseudo-Decompiled Conversion

System: You are given a C source function. Transform it into a pseudo-decompiled representation by performing the following steps. Show intermediate results.

Tools: GETSTRUCTINFO(name, ver), GETMACROVALUE(name, ver).

=== Source Function ===

{source code of f_src}

=== Macro Definitions Referenced by f_src ===

{macros retrieved via GetMacroValue}

=== Build Configurations ===

{compiler flags, conditional compilation defines, target platform}

=== Struct/Type Definitions Referenced by f_src ===

{types retrieved via GetStructInfo}

=== Callee Symbol Classification ===

Imported (preserve name): {list of imported callees}

Exported (preserve name): {list of exported callees}

Internal (strip name): {list of internal callees}

Step 1: Compile-time code expansion. Expand macros, resolve #if/#ifdef, evaluate sizeof/offsetof using definitions above.

Step 2: Compilation optimization. Constant propagation, folding, dead code elimination. Collapse goto-based error handling into nested conditionals. No complex optimizations.

Step 3: Symbol stripping. Strip all variable and parameter names. For callees, preserve imported and exported names, strip internal names. Degrade types to primitives; convert struct accesses to offset-style operations.

Step 4: Stylization. Reformat to match the decompiler style shown below.

=== Decompiler Output Examples ===

{few-shot examples of real decompiled code}

Fig. 5. Prompt template for LLM-based representation conversion.

Figure 4 illustrates the result on DSA_verify (§ 2): optimization collapses goto-based error handling into nested conditionals and restructures the NULL check, symbol stripping replaces identifiers and degrades types, and stylization reformats to IDA conventions. Gaps remain with real decompiled output due to the many-to-many nature of compilation: without knowing the exact compiler, flags, and target architecture, certain details (e.g., stack canary insertion, literal representations) cannot be predicted. Nevertheless, the surface-level approximation suffices: average embedding cosine similarity rises from 0.50 to 0.60 and Recall@20 from 46.1% to 83.3% (§5.3.1).

Step 2: Same-Space Retrieval. The pseudo-decompiled query and all decompiled binary functions (obtained during preprocessing) are encoded into vectors using an embedding model trained on decompiled code [41]. Because both the query and candidates now reside in the same representation space, the embedding distance more faithfully reflects functional similarity. XLoc computes cosine similarity and selects the top- K candidates, excluding any functions already present in C_{rule} , to form the embedding-based candidate set C_{embed} .

3.3 Source-Level Diff-Guided Candidate Verification

Given the candidate sets from recall, the verification stage determines whether any candidate is the true target or the target is absent, through LLM-based reasoning in three steps: *coarse screening* (§ 3.3.1) rapidly filters obviously unrelated candidates, a *tournament* (§ 3.3.2) progressively

Algorithm 1 Source-Level Diff-Guided Candidate Verification

Require: Source function f_{src} , candidate set C , diff database $DiffDB$

Ensure: (true, $addr$) or not_present

```

1: // Coarse Screening
2: for each batch  $b \subseteq C$  do
3:    $C_{fil} \leftarrow C_{fil} \cup \text{LLM-SCREEN}(f_{src}, b)$ 
4: end for
5: if  $C_{fil} = \emptyset$  then
6:   return not_present
7: end if
8: // Tournament with Selective Diff Injection
9: while  $|C_{fil}| > 1$  do
10:  for each pair  $(c_i, c_j) \in \text{PAIR}(C_{fil})$  do
11:    if  $\text{sim}_{lex}(c_i, c_j) > \tau_d$  and  $DiffDB(c_i, c_j) \neq \emptyset$  then
12:       $d \leftarrow DiffDB(c_i, c_j)$ 
13:    else
14:       $d \leftarrow \emptyset$ 
15:    end if
16:     $v \leftarrow \text{LLM-COMPARE}(f_{src}, c_i, c_j, d)$ 
17:    Advance WINNER( $v$ ); eliminate both if  $v = \text{NEITHER}$ 
18:  end for
19:   $C_{fil} \leftarrow \text{survivors}$  {unpaired candidate advances automatically}
20: end while
21: if  $|C_{fil}| = 1$  then
22:   return (true,  $addr(C_{fil}[0])$ )
23: end if
24: return not_present
  
```

eliminates candidates via pairwise comparison, and *diff summary injection* (§ 3.3.3) resolves ambiguities between similar candidates by supplying pre-extracted source-level differential knowledge. Algorithm 1 presents the formal procedure.

3.3.1 Coarse Screening. The first step reduces the candidate set by quickly filtering out functions that are obviously unrelated to the query. XLoc presents the LLM with the source function f_{src} alongside a batch of candidates’ decompiled code. For each candidate, the LLM renders a binary *yes/no* verdict on whether it could possibly implement the same functionality as f_{src} ; candidates receiving a *no* verdict are immediately eliminated. Crucially, this screening is deliberately loose: it only asks whether a candidate *could* implement the same functionality, not whether it *is* the target. This low bar ensures that only clearly unrelated functions are eliminated, while any functionally plausible candidate is preserved for the more precise tournament stage.

This step serves two purposes. Pragmatically, it reduces the number of candidates entering the more expensive pairwise tournament. Conceptually, it provides the first entry point for a “not present” conclusion: if *no* candidate survives coarse screening, XLoc immediately reports that f_{src} is absent from the binary without proceeding to the tournament.

3.3.2 Tournament Verification. When one or more candidates survive coarse screening, XLoc proceeds to rank them. The surviving candidates enter a single-elimination tournament in which pairs are compared head-to-head until one winner remains or all are eliminated.

We adopt pairwise comparison because prior work [40, 61] shows it achieves the highest effectiveness among LLM-based ranking strategies: *pointwise* scoring suffers from unreliable calibration across invocations, while *listwise* ranking is undermined by the “lost in the middle” phenomenon [30] where candidates are overlooked due to position rather than content. Although coarse

screening already reduces candidates to single digits (median 2, mean 4 in our evaluation), the remaining candidates are precisely the hardest to distinguish: they survived screening because they are functionally similar to the target, making finer-grained pairwise comparison necessary to distinguish them.

Tournament flow. In each round, candidates are randomly paired. For each pair (c_i, c_j) , the LLM receives the source code of f_{src} together with the decompiled code of both candidates and produces one of three verdicts:

- c_i wins: c_i more closely matches f_{src} ; c_j is eliminated.
- c_j wins: c_j more closely matches f_{src} ; c_i is eliminated.
- NEITHER: neither candidate matches f_{src} ; both are eliminated.

If the candidate count is odd, the unpaired candidate advances to the next round automatically. Rounds continue until one candidate remains (reported as the match) or all candidates have been eliminated via NEITHER verdicts, in which case XLoc concludes that f_{src} is not present in the binary. This constitutes the second entry point for a “not present” judgment.

3.3.3 Diff Summary Construction and Injection. Diff summaries encode source-level differential knowledge that guides the tournament on *how* to distinguish similar candidates in their decompiled form. Two challenges arise. First, *construction*: identifying potentially confusable similar functions in the source repository and analyzing how they differ from the target in binary-observable terms rather than source-only features that compilation erases. Second, *selective injection*: blindly injecting diff summaries into every pairwise comparison risks misleading the LLM when the two candidates are already distinguishable on their own; injection should be triggered only when the comparison genuinely needs it.

Offline construction. During preprocessing (§ 3.1), XLoc constructs diff summaries in two phases. First, *similar function identification* locates potentially confusable functions in the source repository. Functions that are confusable after compilation typically reside in the same repository and serve analogous roles: they often share naming patterns (e.g., `DSA_verify` vs. `ECDSA_verify`) and exhibit similar code structure. Based on these observations, we apply a two-stage filter that first selects candidates by function-name similarity and then refines the set using lexical code similarity.

We apply a two-stage filter to identify near-neighbors (similar functions) of a given function f . The *name filter* retains candidates g whose function-name edit distance is below a threshold δ_n , selecting functions with related naming patterns. The *code filter* then measures structural similarity. Functions that are confusable after compilation share similar code structure; the features that distinguish them (function names, callee names, type names) are precisely those erased by compilation. We therefore adopt a lexical similarity metric sim_{lex} [4] that mimics this erasure: a C lexer tokenizes each function, stripping comments and whitespace, and normalizing all identifiers and string literals to uniform tokens, so that the comparison reflects purely structural overlap. Let $T(f)$ and $T(g)$ denote the resulting token sequences. The metric iteratively extracts the longest contiguous common token subsequence (*run*) from $T(f)$ and $T(g)$, removes matched tokens after each extraction, and repeats until no run of length $\geq \ell_{\min}$ remains. Let $R_1, R_2, \dots, R_k$ be the extracted runs; the similarity is:

$$\text{sim}_{\text{lex}}(f, g) = \frac{\sum_{i=1}^k |R_i|}{\min(|T(f)|, |T(g)|)} \quad (4)$$

Functions with $\text{sim}_{\text{lex}} \geq \tau_s$ are retained as neighbors (thresholds in § 4).

Second, *diff summary generation* uses an LLM to analyze the source-level differences between f_{src} and each identified neighbor and translate them into binary-observable distinguishing criteria.

Table 1. Diff summary categories. Each difference is classified into one of four types and enriched with context that remains observable in the binary.

Type	Description	Enriched Context	Example
CALL	Different callees	Callee source code	DSA_do_verify vs. ECDSA_do_verify
VALUE	Different constants or strings	Hex & decimal values	0x2A0 vs. 0x2A3
ACCESS	Different struct accesses	Struct definitions	dsa->q vs. eckey->group
OTHER	Other observable differences	–	3 branches vs. 5 branches

For the second phase, the LLM performs a three-step analysis for each neighbor (see Figure 6), with access to three retrieval tools: GETCALLEESOURCE for callee source code, GETSTRUCTINFO for struct declarations, and GETMACROVALUE for recursive macro expansion.

- (1) *Identify raw differences*: read both functions' source code, recursively expanding macros to their final values, and list all functional divergences (different callees, constants, struct accesses, etc.).
- (2) *Filter out non-semantic changes*: discard differences that do not survive compilation, such as variable naming and code style, retaining only functional divergences.
- (3) *Classify and enrich*: categorize each surviving difference into one of four types (Table 1) and enrich it with context for discrimination. For struct access differences, although exact offsets are compiler-dependent and unknown at the source level, providing the struct definition allows the verifier to reason about relative field positions (e.g., the target accesses the 3rd field while the neighbor accesses the 1st).

Each neighbor receives 1–3 key differences, prioritizing directly observable ones (VALUE) over those requiring additional context (CALL) and fine-grained reasoning (ACCESS, OTHER).

All similar functions of f_{src} are presented to the LLM in a single prompt for unified generation; this is practical because similar functions are rare (in our evaluation, 99% of target functions have 0–2 near-neighbors, with the maximum being 4), and joint generation provides a holistic view of the differences. The entire process operates at the source level and is performed offline; the resulting diff database is stored persistently and reused across queries, avoiding redundant LLM invocations. For example, for DSA_verify vs. ECDSA_verify (§ 2), it identifies a CALL difference and an ACCESS difference as the key discriminators (Figure 7).

Online selective injection. Our key observation is that source functions whose implementations are similar also tend to produce similar decompiled code when compiled under the same environment and configuration. This allows XLoc to reuse the same lexical similarity metric sim_{lex} from the offline construction stage on decompiled code to decide, at tournament time, whether a diff summary is needed. This is feasible because sim_{lex} 's underlying tokenizer [4] normalizes all identifiers to a uniform token and compares only code structure (keywords, operators, control flow); since decompiled code is C-like pseudo-code, the tokenizer applies directly, and the anonymous identifiers therein (e.g., sub_FD377, v1) are normalized in exactly the same way as meaningful source-level names. Thus, before each pairwise comparison of candidates c_i and c_j , XLoc computes sim_{lex} over their decompiled code. If the similarity exceeds a threshold τ_d and a precomputed diff summary exists for the corresponding source-function pair, the diff summary is injected into the LLM prompt alongside the decompiled code. The LLM then examines the pair guided by the diff summary, which directs its attention to specific distinguishing features. For example, if the diff summary states that function A calls foo() while function B calls bar(), the LLM examines the call sites in the decompiled code and inspects the callee bodies to determine which function is being invoked, since the original symbol names are typically stripped in the binary. When the similarity

Prompt Template: Diff Summary Generation

System: You are a binary RE expert. After stripping, only these survive: external/library symbols, string constants, numeric immediates, memory access patterns, control flow structure, code size.

Tools: GETCALLEESOURCE(name, ver), GETSTRUCTINFO(name, ver), GETMACROVALUE(name, ver).

=== Target Function: {name(f_src)} ===

{signature + source code of f_src}

=== Similar Function 1: {name(f_1)} ===

{signature + source code of f_1}

...

Step 1: Identify all functional differences (callees, constants, struct accesses, etc.).

Step 2: Filter to binary-observable. Discard: variable names, code style. Retain: external callees or callees with distinguishable binary features; constants; struct differences.

Step 3: Classify each as CALL / VALUE / ACCESS / OTHER; enrich with context via tools. Max 1-3 per neighbor, ranked by discriminative power.

Output format: for each difference, state what differs and how to verify it in a stripped binary.

=== Few-Shot Examples ===

{example diff summaries with expected output format}

Fig. 6. Prompt template for diff summary generation.

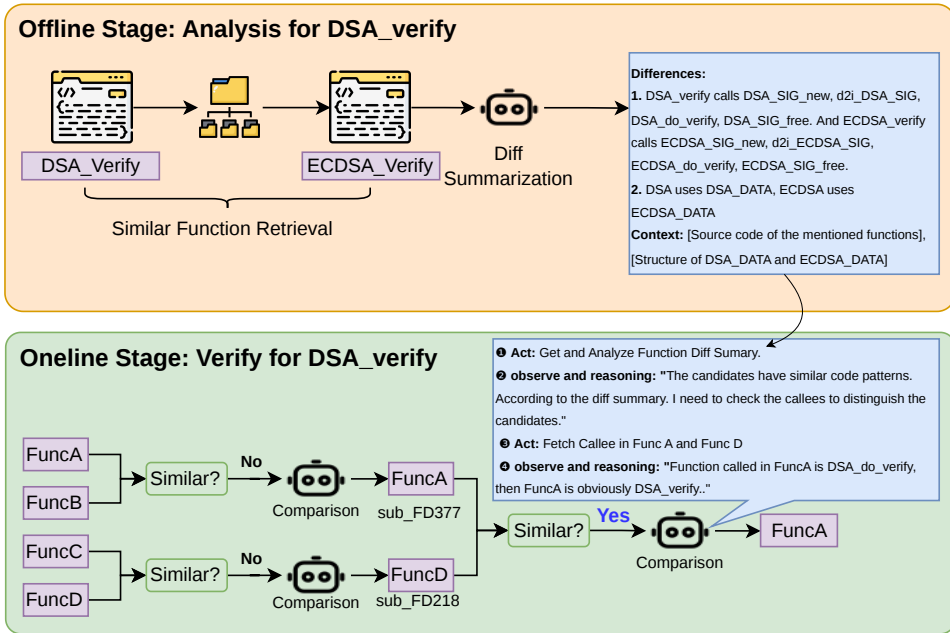


Fig. 7. Running example: locating DSA_verify (CVE-2014-8275) in a stripped OpenSSL binary.

is below the threshold or no diff summary is available, the LLM compares the candidates without additional guidance, as the decompiled code contains sufficient distinguishing information.

3.4 Running Example

We walk through XLoc’s pipeline on the motivation example from § 2: locating `DSA_verify` from CVE-2014-8275 in a stripped OpenSSL binary. Figure 7 illustrates the key steps, divided into an offline analysis phase and an online verification phase.

Offline analysis. During preprocessing, XLoc searches the OpenSSL source repository and identifies `ECDSA_verify` as a near-neighbor of `DSA_verify` based on their high name similarity and lexical code similarity (sim_{lex}). The LLM, equipped with retrieval tools for callee source code, struct declarations, and macro definitions, generates a diff summary identifying two binary-observable differences: a `CALL` difference and a `ACCESS` difference (details in Figure 7).

Online recall and verification. Since `DSA_verify` lacks distinctive strings or preserved symbols, the rule-based pathway produces no candidates. The LLM-bridged pathway then simulates compilation effects on `DSA_verify`’s source to produce a pseudo-decompiled representation, and retrieves top-ranked candidates via same-space embedding similarity. After coarse screening, `FuncA`–`FuncD` (where `FuncA` corresponds to `sub_FD377` (`DSA_verify`) and `FuncD` to `sub_FE218` (`ECDSA_verify`)) enter the tournament. In the first round, two pairwise comparisons are conducted: `FuncA` vs. `FuncB` and `FuncC` vs. `FuncD`. In both matches the LLM eliminates the weaker candidate, advancing `FuncA` and `FuncD` to the next round. In the second round, ❶ sim_{lex} detects that the remaining pair is highly similar, triggering diff summary injection. ❷ Guided by the diff summary, the LLM recognizes that the key distinguishing feature is to check the callees called by them. ❸ It invokes a retrieval tool to fetch the callee bodies of both candidates, and confirms that `FuncA`’s callee matches `DSA_do_verify`, therefore identifying `FuncA` as the correct match for `DSA_verify`.

4 Implementation

XLoc comprises approximately 20,000 lines of Python: 5,000 for preprocessing and 15,000 for the online framework. We use IDA Pro for binary decompilation and Ctags for source code parsing (extracting function boundaries, macro definitions, and type declarations). For code embedding, we use DeJina [41], the state-of-the-art embedding model for decompiled code.

LLM Selection. XLoc uses GPT-5.2 [37] for offline tasks (pseudo-decompilation and diff summary generation), whose results are computed once and reused across queries, prioritizing quality. The online stage (coarse screening, tournament verification) uses GPT-OSS-120B [36], a cost-effective open-source model supporting private deployment.

Prompt Design. All LLM prompts follow a unified template combining chain-of-thought decomposition [48], few-shot examples [32], retrieval tools for specific context, and structured output formats. Figures 5 and 6 show representative templates.

Threshold Selection. For diff summary construction and injection, we use relaxed thresholds to avoid missing true near-neighbors: name edit-distance similarity $\delta_n \geq 0.5$, minimum run length $\ell_{\min} = 24$ (tool default), and lexical similarity $\tau_s = \tau_d = 0.3$ for both neighbor selection and online injection. For verification, we select the top-20 candidates reported by rule-based recall and LLM-bridged recall, respectively.

5 Evaluation

5.1 Experiment Setup

We evaluate XLoc through four research questions:

- **RQ1 (Overall Effectiveness):** How does XLoc compare with existing approaches for source-to-binary function localization?
- **RQ2 (Ablation Study):** How do the core strategies contribute to the overall performance?
- **RQ3 (Efficiency & Cost):** What is the time and token cost of XLoc’s pipeline?

- **RQ4 (Base Model Analysis):** How does the choice of base LLM affect localization performance?

Metrics. The localization task requires a definitive verdict for each query: a confirmed address or an absence conclusion. We therefore need to evaluate two distinct capabilities: whether the system can locate the correct function when it is present (positive cases), and whether it can correctly reject when the function is absent (negative cases).

For positive cases, each query has exactly three possible outcomes, and we define a metric for each: **Positive Accuracy (ACC⁺)**, the percentage where the system returns the correct address; **Wrong Positive Rate (WPR)**, the percentage where the system reports an incorrect address; and **False Negative Rate (FNR)**, the percentage where the system incorrectly concludes “not present.” The three are exhaustive: $ACC^+ + WPR + FNR = 100\%$. For negative cases, we report **Negative Accuracy (ACC⁻)**, the percentage where the system correctly concludes “not present.” When a dataset contains both positive and negative cases, we report the **Harmonic Mean (HM)** of ACC^+ and ACC^- , i.e., $HM = 2 \cdot ACC^+ \cdot ACC^- / (ACC^+ + ACC^-)$, as the overall metric, which avoids bias from class imbalance by requiring both metrics to be simultaneously high.

For the recall-stage evaluation (RQ2), we use **Recall@K (R@K)**, the percentage of queries for which the correct function appears within the top K candidates.

Dataset. While XLoc’s technique is general-purpose, we evaluate it on the important downstream task of locating vulnerable functions in binaries given their source code. We employ two complementary datasets (Table 2). Each test case pairs a *query* (the source code of a vulnerable function) with a target binary; the goal is to determine whether the binary contains the corresponding function and, if so, to identify its address.

The *Lares* dataset [28] is an open-source benchmark originally built for patch presence testing, which we repurpose for function localization. It covers 73 CVEs across 4 open-source projects (157 vulnerable functions, 31 release versions). Binaries are compiled under diverse configurations including three architectures (ARM, x86, x64), four optimization levels (O0–O3), and two compilers (GCC, Clang), yielding 628 binaries. Ground truth is obtained by compiling with debug information to record function addresses, after which all binaries are stripped to remove symbols. All 5,448 test cases are positive (every binary contains the target function), making *Lares* suited for evaluating recall across compilation settings.

The *Debian* dataset is a real-world dataset we construct to additionally evaluate false-positive avoidance. We crawl Debian Security Tracker of the 100 most-installed Debian packages, extract fix-commit links to identify vulnerable functions, and download both pre-patch and post-patch .deb packages. Packages lacking vulnerability records, commit links, extractable vulnerable functions, or available .deb/debug-information packages are excluded. Ground truth is established via DWARF data from the corresponding debug-information packages (–dbgsym/–dbg). Unlike *Lares*, binaries are not stripped but used as-is from the official repositories. Since each package ships multiple binaries, only a subset contain any given vulnerable function; the rest serve as negative cases. The dataset covers 123 CVEs (323 vulnerable functions) across 10 packages and 42 versions, yielding 513 positive and 2,010 negative test cases. All negative cases are same-project binaries with the target query function, and the ratio reflects real-world package distributions.

In summary, *Lares* stress-tests recall and cross-configuration robustness with stripped, positive-only binaries, while *Debian* evaluates precision under realistic conditions with binaries used as-is from official repositories (stripped or partly stripped) and both positive and negative cases.

Baselines. Since function-level source-to-binary matching has limited prior work, we assemble baselines from two complementary directions to cover the solution space: (1) a recent model specifically designed for *compilation-free source-to-binary matching* [55], and (2) *code embedding*

Table 2. Dataset Statistics

Dataset	Composition					Test Cases		
	CVEs	Vuln. Funcs	Projects	Versions	Binaries	Positive	Negative	Total
Lares	73	157	4	31	628	5,448	0	5,448
Debian	123	323	10	42	128	513	2,010	2,523

models that can encode both source and decompiled code for similarity retrieval, ranging from general-purpose to binary-specialized.

- **UniXCoder** [18] is a unified cross-modal pre-trained model for general-purpose code representation, leveraging code, AST, and comments. We use the official release model.
- **Jina-0.5B** [26] is a compact 0.5B-parameter code embedding model optimized for code search and similarity computation. We use the official release model.
- **DeJina** [41] fine-tunes jina-embeddings-v2-base-code [17] on decompiled binary code to bridge the source-to-decompiled representation gap. We follow the original settings and fine-tune it on a BinAtlas subset [41] with no project overlap with our test data.
- **CodeCMR** [55] trains a cross-modal retrieval network that projects source and binary functions into a shared embedding space for direct source-to-binary matching. As the original implementation is not publicly available, we reimplement it following the paper’s design and train on its released dataset.

For the embedding models, the target function source code is embedded as the query and decompiled binary functions serve as retrieval candidates. All baselines participate in RQ1 (overall effectiveness) as end-to-end comparisons with XLoc. In RQ2, they further serve as recall-stage baselines, where we additionally evaluate whether LLM-based pseudo-decompilation of the source query can improve retrieval for the embedding models.

5.2 RQ1: Overall Effectiveness

We compare XLoc against all baselines on two datasets that test complementary aspects of the localization task. On Lares (positive only), every system always reports a top-1 candidate, and we measure whether the reported address is correct. On Debian (positive + negative), a system must also decide whether the target function is present at all. Since similarity-based baselines have no built-in rejection mechanism, we additionally equip them with an optimal similarity threshold τ on Debian: the system reports a match only when the top candidate’s score exceeds τ , and otherwise concludes “not present.” For each baseline, τ is selected to maximize $HM(ACC^+, ACC^-)$ on the Debian dataset, giving baselines the best possible advantage. XLoc uses LLM-based verification rather than similarity scores, so its output is threshold-free by design.

Results on Lares. Table 3 reports results on the Lares dataset, broken down by compilation configuration. Since similarity-based baselines unconditionally return the top-ranked candidate, their errors are entirely misidentification ($WPR = 100\% - ACC^+$, $FNR = 0\%$); the table therefore reports only ACC^+ for baselines. Even the best baseline, DeJina, achieves only 20.0% ACC^+ ; the remaining three baselines all reach at most 12%. In contrast, XLoc achieves **84.4%** ACC^+ (4.2× over DeJina), with 5.0% WPR and 10.6% FNR indicating roughly balanced errors without systematic bias toward either failure mode. Moreover, XLoc’s accuracy is stable across compilation configurations: ACC^+ ranges from 82.4% (ARM) to 85.6% (Clang), a total spread of only 3.2 pp, with 1.9 pp variation across optimization levels and 1.5 pp between compilers, requiring no configuration-specific tuning.

Table 3. Localization performance (%) across compilation configurations on the Lares dataset.

Method		Architecture			Optimization				Compiler		Overall
		ARM	x64	x86	O0	O1	O2	O3	Clang	GCC	
UniXCoder	ACC ⁺	2.9	4.3	3.4	3.4	4.5	3.4	2.8	2.9	3.7	3.5
Jina-0.5B	ACC ⁺	6.1	6.3	4.8	5.4	5.3	4.7	5.8	6.6	5.0	5.3
DeJina	ACC ⁺	22.4	22.2	18.8	22.3	19.3	19.8	18.3	21.2	19.7	20.0
CodeCMR	ACC ⁺	5.3	14.7	12.5	16.9	11.5	9.9	9.3	14.1	11.5	12.0
XLoc	ACC ⁺	82.4	84.1	85.0	85.4	85.1	83.6	83.5	85.6	84.1	84.4
	WPR	5.8	6.1	4.5	4.2	4.8	5.9	5.1	3.8	5.3	5.0
	FNR	11.8	9.8	10.6	10.4	10.1	10.5	11.4	10.6	10.6	10.6

Table 4. Overall localization performance on the Debian dataset. All values are percentages (%). τ^* : optimal similarity threshold maximizing HM(ACC⁺, ACC⁻).

Method	Top-1					With Threshold						
	ACC ⁺	WPR	FNR	ACC ⁻	HM	τ^*	ACC ⁺	WPR	FNR	ACC ⁻	HM	
UniXCoder	6.4	93.6	0.0	0.0	0.0	0.61	5.7	61.4	32.9	65.6	10.4	
Jina-0.5B	11.1	88.9	0.0	0.0	0.0	0.64	11.1	57.1	31.8	58.7	18.7	
DeJina	26.5	73.5	0.0	0.0	0.0	0.62	23.8	52.0	24.2	66.6	35.1	
CodeCMR	12.7	87.3	0.0	0.0	0.0	0.68	12.3	84.8	2.9	42.5	19.1	
XLoc	78.9	6.6	14.4	97.2	87.1	(threshold-free)						

Results on Debian. Table 4 reports results on the Debian dataset, which contains 513 positive and 2,010 negative test cases. Under the Top-1 strategy (left), all similarity-based baselines unconditionally report a match for every query. This yields FNR = 0% and ACC⁻ = 0%: they never miss a positive case, but they also never correctly reject a negative case, resulting in HM = 0% across all baselines. Even on positive cases, the best baseline (DeJina) achieves only 26.5% ACC⁺.

Under the threshold strategy (right), baselines gain the ability to reject queries via a similarity cutoff τ . Figure 8 plots ACC⁺, ACC⁻, and HM as a function of τ for each baseline, revealing a fundamental *threshold dilemma*: raising τ improves ACC⁻ at the expense of ACC⁺, and no single threshold achieves satisfactory performance on both. Even with the oracle-optimal τ^* that maximizes HM on the full test set, the best baseline (DeJina, $\tau^*=0.62$) reaches only HM = 35.1% (ACC⁺ = 23.8%, ACC⁻ = 66.6%), confirming that similarity scores fundamentally lack the discriminative power to separate positive from negative cases. In contrast, XLoc’s LLM-based verification produces a deliberate accept/reject judgment for each query without relying on any score cutoff, achieving 78.9% ACC⁺, 97.2% ACC⁻, and HM = 87.1%, simultaneously excelling at both accurate localization and positive/negative discrimination.

Finding 1. XLoc achieves the highest localization accuracy on both datasets (84.4% ACC⁺ on Lares, 4.2× over the best baseline; 78.9% on Debian), remaining stable across compilation configurations. Unlike similarity-based baselines, which suffer from low accuracy and an inherent threshold dilemma when distinguishing positive from negative cases (best HM = 35.1%), XLoc delivers threshold-free positive/negative discrimination with HM = **87.1%**.

Breakdown analysis. Table 5 breaks down the contribution of each recall step to successful localizations on both datasets. All three steps contribute meaningfully. Embedding recall accounts

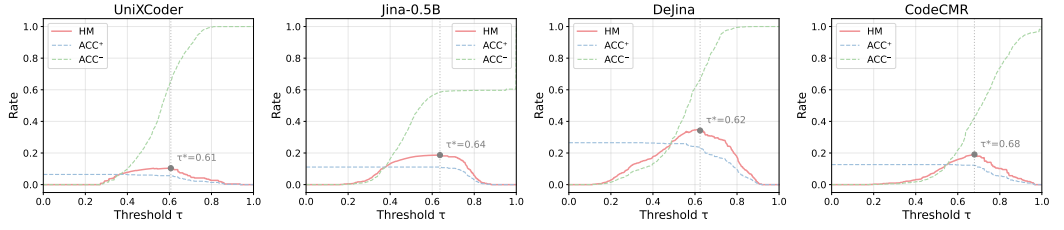


Fig. 8. Effect of similarity threshold τ on ACC^+ , ACC^- , and HM for each baseline on the Debian dataset. Dashed lines mark the optimal τ^* maximizing $HM(ACC^+, ACC^-)$.

Table 5. Contribution of each recall step to true positives (TP).

Pathway	Lares		Debian	
	Count	%	Count	%
Exact match	2,081	45.2	163	40.2
Candidate selection	2,101	45.7	124	30.6
Embedding recall	418	9.1	118	29.1
Total TP	4,600	100.0	405	100.0

Table 6. Recall performance(%) under different representations.

Method	Source Code				Plain LLM				Pseudo-decompiled			
	R@1	R@10	R@20	R@50	R@1	R@10	R@20	R@50	R@1	R@10	R@20	R@50
UniXCoder	3.4	9.7	12.7	17.7	5.6	16.3	20.1	26.6	7.9	18.6	23.9	33.3
Jina-0.5B	5.1	11.7	14.7	20.4	5.5	13.1	16.6	21.6	11.3	22.6	27.3	35.4
CodeCMR	12.0	33.2	41.9	55.2	19.3	44.4	53.7	66.4	25.9	51.5	59.1	71.7
Dejina (used in XLoc)	19.3	40.4	46.1	53.0	31.3	54.2	60.5	69.5	55.1	78.2	83.3	87.1

for a smaller share on Lares (9.1%) not because it is less effective, but because it only activates when both exact match and candidate selection fail; on Debian, where real-world binaries are built with more complex configurations (e.g., higher optimization levels, link-time optimization) that strip more symbols, its share rises to 29.1%. In the verification stage, coarse screening compresses candidates by 67%–76% (from 21.0 to 6.9 on Lares, 9.9 to 2.4 on Debian) while misfiltering the ground truth in only 2.5%/3.8% of cases where it appears in the candidate set. The tournament then correctly identifies the ground truth in 89.8% (Lares) and 83.7% (Debian) of cases where it is present.

5.3 RQ2: Ablation Study

We ablate XLoc’s two core LLM-driven strategies: (1) the *pseudo-decompilation* recall strategy that bridges the source-binary modality gap to improve embedding retrieval, and (2) the *tournament with diff injection* verification strategy that resolves ambiguity among structurally similar candidates.

5.3.1 Recall: LLM-Bridged Pseudo-Decompilation. To understand the effectiveness of our LLM-bridged recall strategy, we evaluate each embedding model under three query representations: (1) original source code, (2) a plain LLM paraphrase without project context or guided steps (ablation



Fig. 9. Similarity distribution between each query representation and the ground-truth decompiled function in DeJina’s embedding space.

control), and (3) the full pseudo-decompiled form with injected context and domain knowledge (§ 3.2.3). Table 6 reports the results on the Lares dataset.

Converting the source query into pseudo-decompiled form consistently improves all embedding models. DeJina benefits the most ($R@1$: 19.3% $\rightarrow$ 55.1%; $R@50$: 53.0% $\rightarrow$ 87.1%), because its decompilation-specific fine-tuning [41] maximally exploits queries presented in the same style. A plain LLM paraphrase without project context yields only modest gains (DeJina $R@1$: 31.3%), whereas the full pseudo-decompilation with guided transformation steps and injected project context (macro definitions, struct layouts, and decompiler output examples; § 3.2.3) nearly triples $R@1$ to 55.1%, confirming that the supplied context and domain knowledge are essential to effective representation conversion.

Figure 9 plots the cosine similarity between each query representation and the ground-truth decompiled function (x86, GCC, O2) in DeJina’s embedding space. Our pseudo-decompiled form achieves the highest average similarity, confirming that the transformation moves queries closer to targets. However, 20 out of 298 cases yield lower similarity, primarily due to small functions with unstable embeddings, symbol preservation mismatches, and aggressive compiler inlining. Since our transformation approximates only the most common compilation patterns, it may not match every specific configuration, though cases that diverge under one setting may benefit under another.

Finding 2. The LLM-bridged pseudo-decompilation strategy effectively narrows the cross-modal gap between source code and decompiled binaries, and is model-agnostic: it consistently boosts all embedding models tested, e.g., DeJina $R@1$ from 19.3% to 55.1%. Moreover, injecting project context and domain knowledge significantly enhances the LLM’s conversion capability.

5.3.2 Verification: Tournament with Diff Injection. We ablate the verification stage to evaluate the individual contribution of its three components: tournament comparison, diff summary injection, and the selective injection strategy. Due to the cost of running the full LLM verification pipeline, the ablation uses a representative Lares subset of 931 cases covering all 73 CVEs, compiled under production-level configurations (O2/O3, ARM/x86, GCC/Clang); minor differences from the full-dataset accuracy in RQ1 are due to the smaller sample.

Table 7 ablates the verification stage to evaluate the contribution of tournament comparison, diff summary injection, and the selective injection strategy. **C** uses only the coarse screening stage (§ 3.3.1), where the LLM scores each candidate’s relevance to the query within batches and reports the highest-scored survivor, without pairwise comparison. Adding **tournament** comparison

Table 7. Ablation study on LLM verification strategies. C: Coarse Screening, T: Tournament, FI: Force Injection, SI: Selective Injection (Ours).

Strategy	ACC ⁺ (%)	WPR(%)	FNR(%)
C (Plain)	75.2	13.3	11.5
C+T	82.1 (↑6.9)	10.6 (↓2.7)	7.3 (↓4.2)
C+T+FI	80.0 (↓2.1)	8.5 (↓2.1)	11.5 (↑4.2)
C+T+SI (XLoc)	85.9 (↑3.8)	7.9 (↓2.7)	6.1 (↓1.2)

(**C+T**, § 3.3.2) raises ACC⁺ by 6.9 pp because pairwise comparison forces the LLM to focus on *relative* differences between two candidates at a time, rather than scoring each against the query in isolation; this contrastive reasoning is more effective at surfacing subtle distinguishing features in decompiled code. Injecting diff summaries into *all* tournament comparisons (**C+T+FI**) actually degrades ACC⁺ by 2.1 pp relative to **C+T** and raises FNR from 7.3% back to 11.5%: when the actual candidates are unrelated to the functions presupposed by the diff summary, the LLM is misled into focusing on irrelevant distinguishing features, causing it to overlook the true matching signals and make incorrect judgments. In contrast, *selective* injection (**C+T+SI**, § 3.3.3) injects diff summaries only when the tournament encounters near-identical candidate pairs (e.g., DSA_verify vs. ECDSA_verify), ensuring that the injected information is relevant to the candidates being compared. This lifts ACC⁺ by 3.8 pp over **C+T** to **85.9%** while keeping both WPR and FNR at their lowest levels, confirming that targeted injection restores discriminative signals precisely where they are needed without misleading the LLM elsewhere.

Detailed analysis of diff injection. To understand the coverage and effectiveness of diff summaries, we examine their construction and runtime behavior. During offline analysis, diff summaries are constructed for 71 out of 157 source functions that have sufficiently similar neighbors in their package. At runtime, 43 of these functions (covering 193 test cases) actually trigger diff injection when the tournament encounters near-identical candidate pairs. The gap between construction (71) and triggering (43) reflects a deliberately conservative strategy: we use a low similarity threshold during offline analysis to ensure diff summaries are pre-computed for all potential confounders, even though many are never triggered because the tournament resolves those cases without difficulty. We analyze the 193 triggered cases and find that selective diff injection raises accuracy from 76.7% (148/193, under tournament alone) to 84.5% (163/193), with improvements in two complementary aspects. First, when the ground truth is present among near-identical candidates, diff summaries provide concrete distinguishing evidence that helps the LLM *correct misidentifications*, flipping 29 cases from incorrect to correct. Second, when the ground truth is absent and both candidates are confusable similar functions, diff summaries help the LLM *reject both incorrect candidates*: among 205 such pairwise comparisons, 188 (91.7%) are correctly rejected, preventing false positives from propagating to subsequent tournament rounds.

Finding 3. Tournament comparison improves ACC⁺ by 6.9 pp through contrastive reasoning. Diff summary injection under a selective strategy effectively distinguishes near-identical candidates, further raising ACC⁺ by 3.8 pp while reducing both WPR and FNR.

5.4 RQ3: Efficiency & Cost

In this RQ we evaluate XLoc’s runtime overhead. The core cost arises from the online stage, since offline source-code analysis and diff summary construction are persisted as a reusable database

Table 8. Average per-case token and time cost of XLoc’s verification pipeline.

Stage	In Tokens	Out Tokens	Total Tokens	Time (s)
Coarse Screening	17,237	2,384	19,621	35.8
Tournament	24,841	3,702	28,543	52.4
Diff Injection	4,461	834	5,295	11.6
Total	46,539	6,920	53,459	99.8

Table 9. Impact of base LLM on XLoc. Mode: I = Instruct, R = Reasoning.

Base Model	Size	Mode	ACC ⁺	WPR	FNR
GPT-OSS-120B	120B	R	85.9	7.9	6.1
DeepSeek-V3.2	671B	I	74.1	2.7	23.2
DeepSeek-V3.2	671B	R	92.4	3.5	4.1
Qwen3-80B-Inst	80B	I	71.5	6.5	21.9
Qwen3-80B-Think	80B	R	82.6	10.2	7.2

and amortized across queries against different binaries. Specifically, we measure the cost of the three recall steps and the three verification steps separately.

In the recall stage, exact match and candidate selection together average 0.5 s per invocation, while embedding recall averages 61.4 s due to LLM-based pseudo-decompilation and vector similarity search. Since 81.4% of cases are resolved by the first two steps alone, the weighted per-case recall cost is approximately $0.5 + 0.186 \times 61.4 \approx 11.9$ s. Table 8 reports the per-case cost of the verification stage, which dominates overall overhead: coarse screening, tournament comparison, and diff verification together consume approximately 53K tokens and 99.8 s on average, with tournament comparison accounting for over half of both. At current vendor pricing [2] (\$0.039 per 1M input tokens / \$0.19 per 1M output tokens) for GPT-OSS-120B, the total cost per case is approximately \$0.003.

Finding 4. XLoc’s online pipeline is practical, averaging approximately 112 s and 53K tokens per query. The verification stage dominates overall cost, while exact match and candidate selection keep per-case recall overhead low by bypassing embedding recall for 81.4% of cases.

5.5 RQ4: Base Model Analysis

To evaluate how XLoc generalizes across base LLMs, we select models along three dimensions: *vendor* (GPT-OSS-120B from OpenAI [3], DeepSeek-V3.2 from DeepSeek [5], Qwen3 from Alibaba [6]), *scale* (80B, 120B, 671B), and *mode* (Instruct vs. Reasoning). All other components remain identical; only the LLM is substituted. Table 9 reports the results.

XLoc achieves $\text{ACC}^+ \geq 71.5\%$ across all configurations, confirming that its design is not tied to a specific model. Two trends emerge. *Reasoning models consistently outperform instruct variants:* DeepSeek-V3.2 improves from 74.1% to 92.4% ACC^+ when switching to reasoning mode; Qwen3-80B shows a similar +11.1 pp gain. The improvement is primarily driven by reduced FNR, indicating that the verification task benefits from explicit chain-of-thought reasoning. *Larger models yield better results:* among reasoning models, DeepSeek-V3.2 (671B) > GPT-OSS-120B > Qwen3 (80B); the same ranking holds for instruct models.

Finding 5. XLoc generalizes across LLMs of different vendors and scales. Reasoning models and larger models consistently achieve higher accuracy, confirming that the task benefits from both chain-of-thought reasoning and greater model capacity.

6 Limitation

We analyzed XLoc’s failure cases on both datasets and identify three primary causes.

Aggressive Compiler Optimizations (FN, WP). Aggressive optimizations such as function inlining can restructure control flow and alter function boundaries, causing the decompiled output to diverge too far from the source for recall or verification to succeed. A subtler case arises when the compiler splits a function into two compilation units (e.g., an entry wrapper and an inlined

body); XLoc may recall one unit while the ground truth labels the other, yielding a wrong positive despite correctly identifying part of the target.

Insufficient Discriminability of Simple/Common Functions (FP, WP). Simple or common functions (e.g., single-line wrappers) lack distinctive features such as unique callees, strings, or constants, making them hard to recall and disambiguate. Even with diff summaries, the distinguishing information lies in data-flow-level details that are largely erased after compilation, leaving insufficient signals for both recall and verification.

Cross-Version Divergence (FN). When the binary incorporates a different version of the library, the source query may no longer represent the binary target accurately, causing recall or verification failures. In our evaluation, XLoc achieves $\sim 91\%$ ACC⁺ on pre-fix binaries but drops to $\sim 78\%$ on post-fix binaries where the patched function has diverged from the query.

7 Related Work

Our work relates to four areas: source-to-binary code matching, binary code similarity detection, source code similarity detection, and LLM-based code analysis.

Source-to-Binary Code Matching. Matching source code against binary code is a fundamental task in software security. At the library level, BAT [20] matches string literals, OSSPolice [12] combines string and structural features, B2SFinder [56] uses syntactic fingerprints, LibAM [27] applies area matching on binary features, LibvDiff [11] performs version-level differential analysis, and BinPro [35] trains source-binary embeddings for library identification. At the function level, CodeCMR [55] trains a cross-modal retrieval network that projects source and binary functions into a shared embedding space, but its accuracy remains limited due to the severe representation gap between source code and compiled binaries. XLIR [16] bridges the gap by compiling source code to LLVM IR and lifting binaries to the same IR, enabling same-space comparison, but still requires compilation on the source side.

Binary Code Similarity Detection. Binary code similarity detection (BCSD) approaches spanning control-flow graph embeddings [15, 29, 52], instruction-level Transformer models [9, 33, 39, 45, 46], emulation-based semantic analysis [23, 47, 60], and input augmentation [49]. Recent work further improves cross-compilation robustness: HermesSim [19] introduces a semantic-oriented graph to reduce inconsistency across compilation settings, and DeJina [41] fine-tunes a source-code embedding model for decompiled code similarity. However, applying BCSD methods to our task requires compiling.

General-Purpose Code Embedding Models. General-purpose code embedding models such as UniXCoder [18] and Jina-Code [26] encode code into dense vectors for semantic search and clone detection. These models are trained on source-code corpora and can accept decompiled pseudocode as input, but perform poorly on cross-modal source-to-binary retrieval due to the distributional gap between source code and decompiled output.

LLMs for Code Analysis. Large language models have been applied to compilation [21, 59], decompilation improvement [7, 22, 53], binary taint analysis [31], and reverse engineering comprehension [38]. These efforts pursue *semantically faithful* translation between source and binary, or direct binary understanding. XLoc differs in that it uses an LLM only for *stylistic simulation* to bridge the retrieval space without requiring semantic correctness, and for tournament-style verification with injected source-level differential knowledge to produce definitive presence/absence judgments.

8 Discussion

8.1 Practical Usage

Downstream Applications. Source-to-binary function localization benefits a range of downstream tasks, including patch presence verification [10, 28, 57], reachability and exploitability analysis [8, 51], SCA false-positive reduction [25, 27, 42, 50], malware analysis [24, 34], plagiarism detection [58], and license compliance auditing [14]. Unlike similarity-based approaches that produce ranked lists requiring manual inspection, XLoc’s definitive verdicts (confirmed address or absence conclusion) enable fully automated pipelines. Its high negative accuracy (97.2% ACC⁻) further suits tasks where false positives are costly, such as license compliance auditing.

Lightweight Private Deployment. We release XLoc as an open-source tool that supports private deployment. Since the offline stage (source-level analysis, diff summary extraction, and pseudo-decompilation) can be pre-computed and cached, the online stage only requires GPT-OSS-120B for verification. Combined with the DeJina embedding model for recall, the entire pipeline fits on a single 40 GB GPU, making XLoc deployable in private or resource-constrained environments without reliance on external APIs.

8.2 Threats to Validity

Internal Validity. XLoc relies on LLMs for multiple pipeline stages, which may introduce non-determinism and data leakage concerns. LLM outputs can vary across runs; we address this by setting the temperature to zero for all LLM calls to ensure reproducibility. The evaluated libraries may overlap with the training data of embedding models and LLMs. We ensured no overlap between DeJina’s fine-tuning set and our test set; other baselines use original weights, so any overlap only inflates their results without affecting our conclusions. For LLMs, our tasks (diff summary, pseudo-decompilation, verification) are straightforward similarity analyses that do not rely on memorized code knowledge, and our ablation (RQ2) confirms that a plain LLM without guided context yields only modest gains.

External Validity. Our evaluation focuses on C/C++ open-source projects, which may raise generalization concerns for other languages or closed-source software. We address this by evaluating on two complementary datasets: Lares covers diverse architectures, optimization levels, and compilers, while Debian provides real-world packages with diverse library versions and compilation environments.

Construct Validity. Ground-truth labels are constructed by matching debug symbols to function addresses, which may introduce labeling noise when aggressive compiler optimizations (e.g., inlining, wrapper generation) break the one-to-one mapping between source and binary functions. We discuss these cases in § 6 and note that such artifacts lead to underestimation rather than overestimation of XLoc’s effectiveness.

9 Conclusion

We presented XLoc, a recall-then-verify framework for source-to-binary function localization that produces definitive verdicts rather than similarity rankings. Its recall stage hierarchically exploits compilation-surviving features for fast matching and bridges the source-binary modality gap via LLM-based pseudo-decompilation without requiring actual compilation; its verification stage disambiguates near-neighbors through tournament-style comparison guided by pre-extracted source-level diff summaries. On two datasets (196 CVEs, 480 vulnerable functions, 756 binaries), XLoc achieves 84.4% localization accuracy (4.2× over the best baseline) and HM = 87.1% for positive/negative discrimination (vs. 35.1%).

10 Data-Availability Statement

The artifact accompanying this paper includes the complete source code of XLoc, the Debian dataset we constructed, and all scripts needed to reproduce the experiments. The Lares dataset is not bundled as it is publicly available [28]. A preliminary anonymized version of the artifact is available at <https://anonymous.4open.science/r/Oopsla26XLoc>. We intend to submit the artifact for Artifact Evaluation following paper notification.

References

- [1] [n. d.]. ISO/IEC 9899:2011 — iso.org. <https://www.iso.org/standard/57853.html>. [Accessed 16-03-2026].
- [2] [n. d.]. Machine Learning Models and Infrastructure | Deep Infra — deepinfra.com. <https://deepinfra.com/>. [Accessed 18-03-2026].
- [3] [n. d.]. OpenAI — openai.com. <https://openai.com/>. [Accessed 17-03-2026].
- [4] [n. d.]. The software and text similarity tester SIM — dickgrune.com. https://dickgrune.com/{P}rograms/similarity_tester/. [Accessed 07-03-2026].
- [5] DeepSeek AI. 2023. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *arXiv preprint arXiv:2401.02954* (2023). <https://arxiv.org/abs/2401.02954>
- [6] Alibaba Group. 2025. *Qwen3: Locally Deployable Large Language Model*. <https://qwen.ai> A locally deployable large language model by Alibaba Cloud.
- [7] Jordi Armengol-Estapé, Jackson Woodruff, Chris Cummins, and Michael F. P. O’Boyle. 2024. SLaDe: A Portable Small Language Model Decompiler for Optimized Assembly. *arXiv preprint arXiv:2305.12520* (2024).
- [8] Peng Deng, Lei Zhang, Yuchuan Meng, Zheming Yang, and Yuan Zhang. 2025. {ChainFuzz}: Exploiting Upstream Vulnerabilities in {Open-Source} Supply Chains. In *34th USENIX Security Symposium (USENIX Security 25)*. 6199–6218.
- [9] Steven HH Ding, Benjamin CM Fung, and Philippe Charland. 2019. Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 472–489.
- [10] Chaopeng Dong, Jingdong Guo, Shouguo Yang, Yang Xiao, Yi Li, Hong Li, Zhi Li, and Limin Sun. 2025. Plocator: Fine-grained patch presence test in binaries via patch code localization. *ACM Transactions on Software Engineering and Methodology* (2025).
- [11] Chaopeng Dong, Siyuan Li, Shouguo Yang, Yang Xiao, Yongpan Wang, Hong Li, Zhi Li, and Limin Sun. 2024. Libvdiff: Library version difference guided oss version identification in binaries. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.
- [12] Ruian Duan, Ashish Bijlani, Meng Xu, Taesoo Kim, and Wenke Lee. 2017. Identifying open-source license violation and 1-day security risk at large scale. In *Proceedings of the 2017 ACM SIGSAC Conference on computer and communications security*. 2169–2185.
- [13] William Enck and Laurie Williams. 2022. Top five challenges in software supply chain security: Observations from 30 industry and government organizations. *IEEE Security & Privacy* 20, 2 (2022), 96–100.
- [14] Muyue Feng, Weixuan Mao, Zimu Yuan, Yang Xiao, Gu Ban, Wei Wang, Shiyang Wang, Qian Tang, Jiahuan Xu, He Su, et al. 2019. Open-source license violations of binary software at large scale. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 564–568.
- [15] Qian Feng, Rundong Zhou, Chengcheng Xu, Yao Cheng, Brian Testa, and Heng Yin. 2016. Scalable graph-based bug search for firmware images. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 480–491.
- [16] Yi Gui, Yao Wan, Hongyu Zhang, Huifang Huang, Yulei Sui, Guandong Xu, Zhiyuan Shao, and Hai Jin. 2022. Cross-language binary-source code matching with intermediate representations. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 601–612.
- [17] Michael Günther, Jackmin Ong, Isabelle Mohr, Alaeddine Abdessalem, Tanguy Abel, Mohammad Kalim Akram, Susana Guzman, Georgios Mastrapas, Saba Sturua, Bo Wang, et al. 2023. Jina embeddings 2: 8192-token general-purpose text embeddings for long documents. *arXiv preprint arXiv:2310.19923* (2023).
- [18] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. *arXiv preprint arXiv:2203.03850* (2022).
- [19] Haojie He, Xingwei Lin, Ziang Weng, Ruijie Zhao, Shuitao Gan, Libo Chen, Yuede Ji, Jiashui Wang, and Zhi Xue. 2024. Code is not natural language: Unlock the power of semantics-oriented graph representation for binary code similarity detection. In *33rd USENIX Security Symposium (USENIX Security 24)*, PHILADELPHIA, PA.
- [20] Armijn Hemel, Karl Trygve Kalleberg, Rob Vermaas, and Eelco Dolstra. 2011. Finding software license violations through binary code clone detection. In *Proceedings of the 8th Working Conference on Mining Software Repositories*.

- 63–72.
- [21] Li Hu, Guoqiang Chen, Xiuwei Shang, Shaoyin Cheng, Benlong Wu, Gangyang Li, Xu Zhu, Weiming Zhang, and Nenghai Yu. 2025. CompileAgent: Automated real-world repo-level compilation with tool-integrated LLM-based agent system. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2078–2091.
 - [22] Peiwei Hu, Ruigang Liang, and Kai Chen. 2024. DeGPT: Optimizing Decompiler Output with LLM. *Proceedings 2024 Network and Distributed System Security Symposium* (2024).
 - [23] Yikun Hu, Hui Wang, Yuanyuan Zhang, Bodong Li, and Dawu Gu. 2019. A semantics-based hybrid approach on binary code similarity comparison. *IEEE Transactions on Software Engineering* 47, 6 (2019), 1241–1258.
 - [24] Jiyong Jang, David Brumley, and Shobha Venkataraman. 2011. Bitshred: feature hashing malware for scalable triage and semantic analysis. In *Proceedings of the 18th ACM conference on Computer and communications security*. 309–320.
 - [25] Ling Jiang, Junwen An, Huihui Huang, Qiyi Tang, Sen Nie, Shi Wu, and Yuqun Zhang. 2024. BinaryAI: Binary Software Composition Analysis via Intelligent Binary Source Code Matching. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
 - [26] Daria Kryvosheieva, Saba Sturua, Michael Günther, Scott Martens, and Han Xiao. 2025. Efficient Code Embeddings from Code Generation Models. arXiv:2508.21290 [cs.CL] <https://arxiv.org/abs/2508.21290>
 - [27] Siyuan Li, Yongpan Wang, Chaopeng Dong, Shouguo Yang, Hong Li, Hao Sun, Zhe Lang, Zuxin Chen, Weijie Wang, Hongsong Zhu, et al. 2023. Libam: An area matching framework for detecting third-party libraries in binaries. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2023), 1–35.
 - [28] Siyuan Li, Yaowen Zheng, Hong Li, Jingdong Guo, Chaopeng Dong, Chunpeng Yan, Weijie Wang, Yimo Ren, Limin Sun, and Hongsong Zhu. 2025. Lares: LLM-driven Code Slice Semantic Search for Patch Presence Testing. *arXiv preprint arXiv:2511.01252* (2025).
 - [29] Yujia Li, Chenjie Gu, Thomas Dullien, Oriol Vinyals, and Pushmeet Kohli. 2019. Graph matching networks for learning the similarity of graph structured objects. In *International conference on machine learning*. PMLR, 3835–3845.
 - [30] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranajape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics* 12 (2024), 157–173.
 - [31] Puzhuo Liu, Chengnian Sun, Yaowen Zheng, Xuan Feng, Chuan Qin, Yuncheng Wang, Zhenyang Xu, Zhi Li, Peng Di, Yu Jiang, et al. 2025. Llm-powered static binary taint analysis. *ACM Transactions on Software Engineering and Methodology* (2025).
 - [32] Ben Mann, Nick Ryder, Melanie Subbiah, J Kaplan, P Dhariwal, A Neelakantan, P Shyam, G Sastry, A Askell, S Agarwal, et al. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* 1, 3 (2020), 3.
 - [33] Luca Massarelli, Giuseppe Antonio Di Luna, Fabio Petroni, Roberto Baldoni, and Leonardo Querzoni. 2019. Safe: Self-attentive function embeddings for binary similarity. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 16th International Conference, DIMVA 2019, Gothenburg, Sweden, June 19–20, 2019, Proceedings 16*. Springer, 309–329.
 - [34] Reza Mirzazadeh, Mohammad Hossein Moattar, and Majid Vafaei Jahan. 2015. Metamorphic malware detection using linear discriminant analysis and graph similarity. In *2015 5th International Conference on Computer and Knowledge Engineering (ICCKE)*. IEEE, 61–66.
 - [35] Dhaval Miyani, Zhen Huang, and David Lie. 2017. Binpro: A tool for binary source code provenance. *arXiv preprint arXiv:1711.00830* (2017).
 - [36] OpenAI. 2025. *GPT-5 Nano*. <https://openai.com> A locally deployable large language model by OpenAI.
 - [37] OpenAI. 2025. *GPT-5.2*. <https://openai.com> A frontier model for complex professional work by OpenAI.
 - [38] Hammond Pearce, Benjamin Tan, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri, and Brendan Dolan-Gavitt. 2022. Pop Quiz! Can a Large Language Model Help With Reverse Engineering? *arXiv preprint arXiv:2202.01142* (2022).
 - [39] Kexin Pei, Zhou Xuan, Junfeng Yang, Suman Jana, and Baishakhi Ray. 2020. Trex: Learning execution semantics from micro-traces for binary similarity. *arXiv preprint arXiv:2012.08680* (2020).
 - [40] Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, et al. 2024. Large language models are effective text rankers with pairwise ranking prompting. In *Findings of the Association for Computational Linguistics: NAACL 2024*. 1504–1518.
 - [41] Jingyi Shi, Yufeng Chen, Yang Xiao, Yuekang Li, Zhengzi Xu, Sihao Qiu, Chi Zhang, Keyu Qi, Yeting Li, Xingchu Chen, et al. 2025. A Large Scale Study of AI-based Binary Function Similarity Detection Techniques for Security Researchers and Practitioners. *arXiv preprint arXiv:2511.01180* (2025).
 - [42] Wei Tang, Yanlin Wang, Hongyu Zhang, Shi Han, Ping Luo, and Dongmei Zhang. 2022. Libdb: An effective and efficient framework for detecting third-party libraries in binaries. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 423–434.

- [43] Wei Tang, Zhengzi Xu, Chengwei Liu, Jiahui Wu, Shouguo Yang, Yi Li, Ping Luo, and Yang Liu. 2022. Towards understanding third-party library dependency in c/c++ ecosystem. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.
- [44] Hao Wang, Zeyu Gao, Chao Zhang, Zihan Sha, Mingyang Sun, Yuchen Zhou, Wenyu Zhu, Wenju Sun, Han Qiu, and Xi Xiao. 2024. CLAP: Learning Transferable Binary Code Representations with Natural Language Supervision. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 503–515.
- [45] Hao Wang, Zeyu Gao, Chao Zhang, Mingyang Sun, Yuchen Zhou, Han Qiu, and Xi Xiao. 2024. CEBin: A Cost-Effective Framework for Large-Scale Binary Code Similarity Detection. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 149–161.
- [46] Hao Wang, Wenjie Qu, Gilad Katz, Wenyu Zhu, Zeyu Gao, Han Qiu, Jianwei Zhuge, and Chao Zhang. 2022. jtrans: Jump-aware transformer for binary code similarity. *arXiv preprint arXiv:2205.12713* (2022).
- [47] Shuai Wang and Dinghao Wu. 2017. In-memory fuzzing for binary code similarity analysis. *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2017), 319–330.
- [48] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [49] Wai Kin Wong, Huaijin Wang, Zongjie Li, and Shuai Wang. 2024. BinAug: Enhancing Binary Similarity Analysis with Low-Cost Input Repairing. *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)* (2024), 51–63.
- [50] Jiahui Wu, Zhengzi Xu, Wei Tang, Lyuye Zhang, Yueming Wu, Chengyue Liu, Kairan Sun, Lida Zhao, and Yang Liu. 2023. Ossfp: Precise and scalable c/c++ third-party library detection using fingerprinting functions. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 270–282.
- [51] Yulun Wu, Zeliang Yu, Ming Wen, Qiang Li, Deqing Zou, and Hai Jin. 2023. Understanding the threats of upstream vulnerabilities to downstream projects in the maven ecosystem. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*.
- [52] Xiaojun Xu, Chang Liu, Qian Feng, Heng Yin, Le Song, and Dawn Song. 2017. Neural network-based graph embedding for cross-platform binary code similarity detection. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 363–376.
- [53] Xiangzhe Xu, Zhuo Zhang, Shiwei Feng, Yapeng Ye, Zian Su, Nan Jiang, Siyuan Cheng, Lin Tan, and Xiangyu Zhang. 2023. LmPa: Improving Decompilation by Synergy of Large Language Model and Program Analysis. *arXiv preprint arXiv:2306.02546* (2023).
- [54] Can Yang, Zhengzi Xu, Hongxu Chen, Yang Liu, Xiaorui Gong, and Baoxu Liu. 2022. ModX: binary level partially imported third-party library detection via program modularization and semantic matching. In *Proceedings of the 44th International Conference on Software Engineering*. 1393–1405.
- [55] Zeping Yu, Wenxin Zheng, Jiaqi Wang, Qiye Tang, Sen Nie, and Shi Wu. 2020. Codecmr: Cross-modal retrieval for function-level binary source code matching. *Advances in Neural Information Processing Systems* 33 (2020), 3872–3883.
- [56] Zimu Yuan, Muyue Feng, Feng Li, Gu Ban, Yang Xiao, Shiyang Wang, Qian Tang, He Su, Chendong Yu, Jiahuan Xu, et al. 2019. B2sfinder: Detecting open-source software reuse in cots software. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1038–1049.
- [57] Qi Zhan, Xing Hu, Zhiyang Li, Xin Xia, David Lo, and Shanping Li. 2024. Ps3: Precise patch presence test based on semantic symbolic signature. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.
- [58] Fangfang Zhang, Dinghao Wu, Peng Liu, and Sencun Zhu. 2014. Program logic based software plagiarism detection. In *2014 IEEE 25th international symposium on software reliability engineering*. IEEE, 66–77.
- [59] Hongbin Zhang, Shihao Gao, Yang Liu, Mingjie Xing, Yanjun Wu, and Chen Zhao. 2025. Exploring the Feasibility of End-to-End Large Language Model as a Compiler. In *2025 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.
- [60] Anshunkang Zhou, Yikun Hu, Xiangzhe Xu, and Charles Zhang. 2024. ARCTURUS: Full Coverage Binary Similarity Analysis with Reachability-Guided Emulation. *ACM Transactions on Software Engineering and Methodology* 33, 4 (2024), 1–31.
- [61] Shengyao Zhuang, Honglei Zhuang, Bevan Koopman, and Guido Zuccon. 2024. A Setwise Approach for Effective and Highly Efficient Zero-Shot Ranking with Large Language Models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 38–47.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009